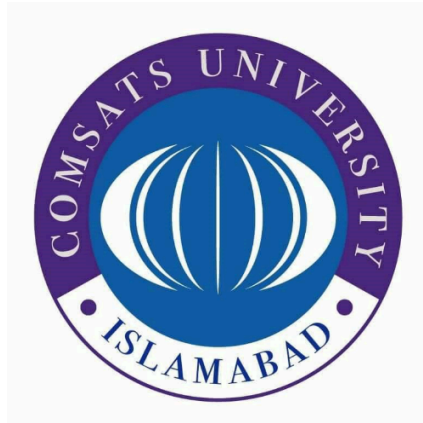




PARALLEL & DISTRIBUTED COMPUTING

LAB ASSIGNMENT #8

PREPARED BY

M Mukaram Awan
SP22-BCS-141

SUBMITTED TO

Sir Akhzar Nazeer

API Communication, Microservices & Performance Comparison

Objective

Implement REST, tRPC-style, and gRPC communication, including service-to-service gRPC. Compare performance, payload size, and latency.

1. REST API

REST server built with Flask. Client measures response time and payload size.

rest_server.py

```
from flask import Flask, jsonify
import json

app = Flask(__name__)
users = json.load(open("users.json"))

@app.get("/user/<int:id>")
def get_user(id):
    for u in users:
        if u["id"] == id:
            return jsonify(u)
    return jsonify({"error": "not found"}), 404

app.run(port=3000)
```

rest_client.py

```
import requests, time

t0 = time.time()
r = requests.get("http://localhost:3000/user/1")
t = (time.time() - t0) * 1000

print("REST TIME:", round(t, 2), "ms")
print("PAYLOAD SIZE:", len(r.text))
```

```
print("RESPONSE:", r.text)
```

2. tRPC-style API

trpc_server.py

```
from fastapi import FastAPI
from pydantic import BaseModel
import json

app = FastAPI()
users = json.load(open("users.json"))

class Input(BaseModel):
    id: int

@app.post("/trpc/getUser")
def get_user(input: Input):
    for u in users:
        if u["id"] == input.id:
            return u
    return {"error": "not found"}
```

trpc_client.py

```
import requests, time

t0 = time.time()
r = requests.post("http://localhost:4001/trpc/getUser", json={"id": 1})
t = (time.time() - t0) * 1000

print("tRPC TIME:", round(t, 2), "ms")
print("PAYLOAD SIZE:", len(r.text))
print("RESPONSE:", r.text)
```

3. gRPC API

user.proto

```
syntax = "proto3";

message UserRequest {
  int32 id = 1;
}

message User {
  int32 id = 1;
  string name = 2;
  string email = 3;
}

service UserService {
  rpc GetUser(UserRequest) returns (User);
}
```

grpc_server.py

```
import grpc
import json
import time
from concurrent import futures

import user_pb2
import user_pb2_grpc

users = json.load(open("users.json"))

class UserService(user_pb2_grpc.UserServiceServicer):
    def GetUser(self, request, context):
        for u in users:
            if u["id"] == request.id:
                return user_pb2.User(id=u["id"], name=u["name"], email=u["email"])
```

```

        return user_pb2.User()

server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
user_pb2_grpc.add_UserServiceServicer_to_server(UserService(), server)
server.add_insecure_port("0.0.0.0:50051")
server.start()
print("gRPC server running")
server.wait_for_termination()

```

grpc_client.py

```

import grpc, time
import user_pb2
import user_pb2_grpc

channel = grpc.insecure_channel("localhost:50051")
client = user_pb2_grpc.UserServiceStub(channel)

t0 = time.time()
resp = client.GetUser(user_pb2.UserRequest(id=1))
t = (time.time() - t0) * 1000

print("gRPC TIME:", round(t, 2), "ms")
print("BINARY SIZE:", len(resp.SerializeToString()))
print("RESPONSE:", resp)

```

4. Service-to-Service gRPC

service_b.py

```

from flask import Flask, jsonify
import grpc
import user_pb2
import user_pb2_grpc

app = Flask(__name__)

```

```

channel = grpc.insecure_channel("localhost:50051")
client = user_pb2_grpc.UserServiceStub(channel)

@app.get("/api/getUser/<int:id>")
def get_user(id):
    resp = client.GetUser(user_pb2.UserRequest(id=id))
    return jsonify({"id": resp.id, "name": resp.name, "email": resp.email})

app.run(port=3002)

```

Comparison Summary

Technology	Serialization	Latency	Payload Size	Scalability	Use Case
REST	JSON	High	Large	Medium	Simple apps
tRPC-style	JSON	Medium	Medium	Medium	Typed RPC
gRPC	Binary	Low	Small	High	Microservices
gRPC S2S	Binary	Lowest	Smallest	High	Service-to-service